API contracts

What each call does, with a worked example. 11 November 2026.

> The examples follow Ada (`ada`, `ada@studio.test`, id `7c2a9e14-1b0d-4e6a-9f31-0a8b6c4d2e11`) and a file `notes.pdf`. She later invites Sam (`sam@studio.test`) into the workspace Studio (`f0e1d2c3-4a5b-4678-9abc-def012345678`). File calls are used by Drive and the website. People calls are the website only. A workspace call also sends the header `X-Workspace-Id: f0e1d2c3-4a5b-4678-9abc-def012345678`.

Ada's My Drive is namespace `a1f0c3d2-88e4-4b1a-9c77-2d6e5b0a4410`, root folder `3e8b1c40-6f2a-4d19-8a55-91c0e7b34d02`. The folder Drawings in Drive is node `5d6c8a20-4e17-4b8f-a903-1c2b7d9e6f40`. The project of the same name, used when inviting someone, is group number `42`. The file `notes.pdf` is `9b4e2a71-0c58-4f93-b612-7d1a8e05c334`.

SIGN-IN

POST `/login`

This is sign-in. `userName` may be the username or the email. An ordinary account gets a token. An account with a second step gets a challenge and no token yet. Drive keeps `token` and `user`. The website also uses `workspaces` and `apps`.

EXAMPLE

```
{ "userName": "ada", "password": "correct-horse" }

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.signature",
  "user": {
    "id": "7c2a9e14-1b0d-4e6a-9f31-0a8b6c4d2e11",
    "email": "ada@studio.test",
    "name": "ada",
    "fullName": "Ada Novak",
    "role": "Creator",
    "apps": { "works": true, "connector": true, "plan": true, "drive": true },
    "driveQuotaBytes": 5368709120
  },
  "canCreateWorkspace": false,
  "totpEnabled": false,
  "workspaces": []
}
```

A second step looks like `{ "requiresTwoFactor": true, "method": "email", "loginChallenge": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.challenge", "email": "ada@studio.test" }`. `method` is `email` or `totp`.

Who uses it. Drive sign-in (`DriveAppModel.swift`) and the website (`space-admin/src/lib/auth.tsx`). Server: `space-api/src/routes/auth.ts`.

POST `/login/2fa`

This finishes sign-in. The code is 6 to 8 characters. The answer is the same shape as an ordinary sign-in.

```
{ "loginChallenge": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.challenge", "code": "482913" }
```

Who uses it. Drive and the website login. Server: `space-api/src/routes/auth.ts`.

THIS COMPUTER

POST `/me/apps/tokens`

After sign-in, Drive asks for a token that belongs to this Mac. `app` is `drive.space`, `connector.space`, or `collab.space`. `expiresAt` is 12 hours away. `pairingExpiresAt` is 90 days away.

```
{ "app": "drive.space", "name": "Ada's MacBook" }

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.app",
  "app": "drive.space",
  "id": "c0ffee00-1111-4222-8333-444455556666",
  "expiresAt": "2026-11-11T22:00:00.000Z",
  "pairingExpiresAt": "2027-02-09T10:00:00.000Z"
}
```

Who uses it. Drive (`DriveAppModel.swift`). Website: `LocalAppsPage.tsx` and `ComputerSettingsSection.tsx`. Server: `space-api/src/routes/deviceApps.ts`.

POST `/me/apps/tokens/refresh`

Drive sends the current Mac token and an empty body `{}`. The answer is a new token in the same shape. Drive replaces the keychain token.

Who uses it. Drive (`DriveAppModel.swift`, `DriveAPI.swift`). Server: `space-api/src/routes/deviceApps.ts`.

DELETE `/me/apps/tokens/:tokenId`

No body. The server answers 204 and that Mac token stops working. Example path: `/me/apps/tokens/c0ffee00-1111-4222-8333-444455556666`.

Who uses it. Drive sign-out and the website local-apps list. Server: `space-api/src/routes/deviceApps.ts`.

FILES AND FOLDERS

GET `/storage/listing`

No parameters. The answer is the drives Ada can open. `kind` is `personal`, `org`, or `workspace`.

```
{
  "namespaces": [{
    "id": "a1f0c3d2-88e4-4b1a-9c77-2d6e5b0a4410",
    "kind": "personal",
    "title": "My Drive",
    "userId": "7c2a9e14-1b0d-4e6a-9f31-0a8b6c4d2e11",
    "organizationId": null,
    "workspaceId": null,
    "rootNodeId": "3e8b1c40-6f2a-4d19-8a55-91c0e7b34d02"
  }],
  "roots": [{
    "id": "3e8b1c40-6f2a-4d19-8a55-91c0e7b34d02",
    "name": "My Drive",
    "kind": "folder",
    "role": "owner",
    "byteSize": 0
  }]
}
```

Who uses it. Finder (`DriveItemSource.swift`, `DomainManager.swift`) and `DrivePage.tsx`. Server: `space-api/src/routes/storage.ts`.

GET `/storage/nodes/:id`

One file or folder. `role` is `viewer`, `editor`, or `owner`. `byteSize` is bytes. `contentHash` is the SHA-256 of the bytes, 64 hex characters.

```
GET /storage/nodes/9b4e2a71-0c58-4f93-b612-7d1a8e05c334

{
  "id": "9b4e2a71-0c58-4f93-b612-7d1a8e05c334",
  "namespaceId": "a1f0c3d2-88e4-4b1a-9c77-2d6e5b0a4410",
  "parentId": "5d6c8a20-4e17-4b8f-a903-1c2b7d9e6f40",
  "name": "notes.pdf",
  "kind": "file",
  "appKind": "file",
  "mime": "application/pdf",
  "byteSize": 24831,
  "contentHash": "9a36cfbf7a720bd5699bf9044ca07b5a7ad0eb7d00d3e19110bd2a08b23f9f8f",
  "readOnly": false,
  "cloudOnly": false,
  "role": "owner",
  "isPersonal": true,
  "trashedAt": null,
  "createdAt": "2026-11-11T09:12:00.000Z",
  "updatedAt": "2026-11-11T10:04:00.000Z"
}
```

Who uses it. Finder and the website Drive page. Server: `space-api/src/routes/storage.ts`.

GET `/storage/nodes/:id/children`

The inside of a folder. This is Drawings with `notes.pdf` in it.

```
GET /storage/nodes/5d6c8a20-4e17-4b8f-a903-1c2b7d9e6f40/children

{
  "parent": { "id": "5d6c8a20-4e17-4b8f-a903-1c2b7d9e6f40", "name": "Drawings", "kind": "folder", "role": "owner" },
  "children": [
    { "id": "9b4e2a71-0c58-4f93-b612-7d1a8e05c334", "name": "notes.pdf", "kind": "file", "byteSize": 24831, "role": "owner" }
  ]
}
```

Who uses it. Finder and the website Drive page and editor. Server: `space-api/src/routes/storage.ts`.

GET `/storage/changes`

The Drive sync log. `namespaceId` is the drive. `cursor` is the last row Drive has applied. The server sends up to 500 newer rows. `op` is `create`, `rename`, `move`, `content`, `trash`, `delete`, `restore`, or `share`. Drive treats `trash`, `delete`, and `purge` as “take this out of Finder” and every other value as “refresh this item.”

```
GET /storage/changes?namespaceId=a1f0c3d2-88e4-4b1a-9c77-2d6e5b0a4410&cursor=1842

{
  "changes": [{
    "cursor": "1843",
    "namespaceId": "a1f0c3d2-88e4-4b1a-9c77-2d6e5b0a4410",
    "nodeId": "9b4e2a71-0c58-4f93-b612-7d1a8e05c334",
    "op": "rename",
    "actorUserId": "7c2a9e14-1b0d-4e6a-9f31-0a8b6c4d2e11",
    "createdAt": "2026-11-11T10:04:00.000Z"
  }]
}
```

Who uses it. Drive only, from `DomainManager.swift`. Server: `listChanges` in `space-api/src/lib/storage/service.ts`.

POST `/storage/nodes`

Creates a folder or an empty file. `kind` is `folder`, `file`, or `project`. Left out, it is `folder`. The answer is 201, and the sync log gets `create`.

```
{ "parentId": "3e8b1c40-6f2a-4d19-8a55-91c0e7b34d02", "name": "Drawings", "kind": "folder" }

{ "id": "5d6c8a20-4e17-4b8f-a903-1c2b7d9e6f40", "name": "Drawings", "kind": "folder", "role": "owner", "byteSize": 0 }
```

Who uses it. Finder (`DriveSync.swift`) and the website Drive page. Server: `space-api/src/routes/storage.ts`.

PATCH `/storage/nodes/:id`

Renames an item. The only field is the new name, 1 to 256 characters. The sync log gets `rename`.

```
PATCH /storage/nodes/9b4e2a71-0c58-4f93-b612-7d1a8e05c334
{ "name": "Brief.pdf" }
```

Who uses it. Finder and the website Drive page. Server: `space-api/src/routes/storage.ts`.

POST `/storage/nodes/:id/move`

Moves an item. `parentId` is the folder it should land in. Drive sends only that. The server also accepts an optional new `name`. The sync log gets `move`.

```
POST /storage/nodes/9b4e2a71-0c58-4f93-b612-7d1a8e05c334/move
{ "parentId": "5d6c8a20-4e17-4b8f-a903-1c2b7d9e6f40" }
```

Who uses it. Finder and the website Drive page. Server: `space-api/src/routes/storage.ts`.

DELETE `/storage/nodes/:id`

Puts the file or folder in the trash. No body. The server answers 204 and writes `trash`. The record then has `trashedAt`, for example `2026-11-11T10:20:00.000Z`.

Who uses it. Finder and the website Drive page. Server: `space-api/src/routes/storage.ts`.

PUT `/storage/nodes/:id/content`

The body is the file itself. The content type is the file's type, such as `application/pdf`. The answer is the updated record, with a new `byteSize` and `contentHash`. The sync log gets `content`.

```
PUT /storage/nodes/9b4e2a71-0c58-4f93-b612-7d1a8e05c334/content
Content-Type: application/pdf
(the 24,831 bytes)

{ "id": "9b4e2a71-0c58-4f93-b612-7d1a8e05c334", "byteSize": 24831, "contentHash": "9a36cfbf7a720bd5699bf9044ca07b5a7ad0eb7d00d3e19110bd2a08b23f9f8f" }
```

Who uses it. Finder on save and the website Drive page on upload. Server: `space-api/src/routes/storage.ts`.

GET `/storage/nodes/:id/content`

Downloads the bytes. Headers include `Content-Type: application/pdf`, `Content-Length: 24831`, and `Content-Disposition: attachment; filename="notes.pdf"; filename*=UTF-8''notes.pdf`.

Who uses it. Finder and the website Drive page. Server: `space-api/src/routes/storage.ts`.

GET `/storage/nodes/:id/open`

Says how to open an item. A scene returns a web address and a `space://` link. An ordinary file returns `download` pointing at the download call. Drive defines `openInfo` and nothing calls it yet.

```
GET /storage/nodes/8a1d0e55-77b2-4c30-9d14-6f2e0b9a1001/open

{
  "node": { "id": "8a1d0e55-77b2-4c30-9d14-6f2e0b9a1001", "name": "Lobby.skyscene", "appKind": "scene", "cloudOnly": true, "extension": ".skyscene" },
  "spaceUrl": "https://app.skyform.space/scenes/editor?sceneId=8a1d0e55-77b2-4c30-9d14-6f2e0b9a1001&nodeId=8a1d0e55-77b2-4c30-9d14-6f2e0b9a1001",
  "href": "space://open?node=8a1d0e55-77b2-4c30-9d14-6f2e0b9a1001",
  "download": null
}
```

Who uses it. Server: `space-api/src/routes/storage.ts`. Client method: `DriveAPI.swift` `openInfo`.

SHARING A COPY

POST `/storage/dispatches`

`nodeIds` is 1 to 200 item ids. The answer is one JSON line at a time. `event` is `start`, then `progress`, then `done`. `publicId` is a short token. The page is `/d/` plus that token.

```
{ "nodeIds": ["9b4e2a71-0c58-4f93-b612-7d1a8e05c334"] }

{"event":"start","id":"b7e1c290-4a55-4d18-9e02-
6c8f1a0d3344","publicId":"k3mP9xQa2LcB","title":"notes.pdf","url":"https://app.skyform.space/d/k3mP9xQa2LcB","fileCount":1,"totalBytes":24831,"copiedBytes":0}
{"event":"progress","name":"notes.pdf","index":1,"fileCount":1,"copiedBytes":24831,"totalBytes":24831}
{"event":"done","publicId":"k3mP9xQa2LcB","url":"https://app.skyform.space/d/k3mP9xQa2LcB","copiedBytes":24831,"totalBytes":24831}
```

A failure ends with `{"event":"error","message":"Could not create this dispatch."}`.

Who uses it. Drive `DriveDispatchPanel.swift` and website `DriveDispatchPanel.tsx`. Server: `space-api/src/lib/storage/dispatch.ts`.

POST `/storage/dispatches/:id/notify`

Emails the share link. The path id is the dispatch id from `start`, not the public token.

```
POST /storage/dispatches/b7e1c290-4a55-4d18-9e02-6c8f1a0d3344/notify
{ "email": "sam@studio.test" }

{ "ok": true, "delivered": true, "message": "Sent to sam@studio.test" }
```

When mail is not configured, `delivered` is false and `message` is `"The link is ready, but mail is not configured on this server."`

Who uses it. The Drive panel and the website dispatch board. Server: `space-api/src/routes/storage.ts`.

PEOPLE IN A WORKSPACE

These calls are how the website lists, invites, and removes people in Studio. They do not run in Drive. Each one sends `X-Workspace-Id: f0e1d2c3-4a5b-4678-9abc-def012345678` except the public invite link and "my invites." A role is `Owner`, `Manager`, `Creator`, `Viewer`, or `Customer`. The permission code is those rights joined with `|`, sorted. A Viewer code is `planning.chat|planning.view|scenes.load|sessions.view`.

GET `/users`

The people who belong to Studio. The answer is a list, not wrapped in an object. Each person has their name, email, role, and the projects they can open. The caller needs the right to view users.

```
[
  {
    "id": "7c2a9e14-1b0d-4e6a-9f31-0a8b6c4d2e11",
    "name": "ada",
    "fullName": "Ada Novak",
    "email": "ada@studio.test",
    "role": "Owner",
    "rolePreset": "owner",
    "permissionCode": "assets.edit|assets.view|groups.edit|users.edit|workspace.edit",
    "description": "",
    "isPlatformAdmin": false,
    "hasAvatar": true,
    "startupSceneAssetId": null,
    "groupAccess": [{ "id": 42, "name": "Drawings", "parentGroupId": null, "isPersonal": false }]
  }
]
```

The owner code above is shortened. A real owner code lists every right the product knows, still joined with `|`.

Who uses it. The people page, `space-admin/src/pages/UsersPage.tsx`, through `space-admin/src/lib/users.ts`. Server: `space-api/src/routes/users.ts`.

GET `/users/:id`

One person in this workspace, in the same shape as a row of the list. Unknown here means they are not a member of Studio, even if they have an account.

```
GET /users/2b6c8d90-1111-4222-8333-555566667777
X-Workspace-Id: f0e1d2c3-4a5b-4678-9abc-def012345678
```

Who uses it. The person page at `/users/:id`. Server: `space-api/src/routes/users.ts`.

GET `/users/my`

The signed-in person, as a member of the current workspace. Same shape as one row above. The website uses this for “me in this workspace,” including which projects Ada can open and her startup scene.

Who uses it. `space-admin/src/lib/users.ts` `usersApi.my`. Server: `space-api/src/routes/users.ts`.

GET `/users/:id/avatar`

The person’s picture, as image bytes. No JSON. A person with no picture is a 404.

Who uses it. `space-admin/src/components/UserAvatar.tsx`. Server: `space-api/src/routes/users.ts`.

PATCH `/users/:id`

Changes Sam’s place in Studio: the role, the permission code, or the startup scene. It does not change Sam’s name, email, or password. Those belong on the profile. Ada cannot change her own role here. A Customer must already have a startup scene, or this call must send one. The answer is Sam’s updated member record.

```
PATCH /users/2b6c8d90-1111-4222-8333-555566667777
{
  "role": "Viewer",
  "permissionCode": "planning.chat|planning.view|scenes.load|sessions.view",
  "startupSceneAssetId": null
}
```

Who uses it. The people page, through `usersApi.update`. Server: `space-api/src/routes/users.ts`.

DELETE `/users/:id`

Takes Sam out of Studio. Sam’s account, password, and My Drive stay. Ada cannot remove herself. The server answers 204, with an empty body, and emails Sam that they were removed. Their personal folders inside this workspace go away.

```
DELETE /users/2b6c8d90-1111-4222-8333-555566667777
X-Workspace-Id: f0e1d2c3-4a5b-4678-9abc-def012345678
```

Who uses it. The people page, `UsersPage.tsx`, through `usersApi.remove`. Server: `space-api/src/routes/users.ts`.

POST `/users`

Creating a user directly is turned off. The server answers 410 and tells the website to invite them instead.

```
{ "message": "Direct user creation is disabled. Invite users via POST /workspaces/:id/invites" }
```

Who uses it. Older clients. The people page now calls the invite route. Server: `space-api/src/routes/users.ts`.

INVITING SOMEONE

POST `/workspaces/:workspaceId/invites`

Ada invites Sam into Studio. The email must be a real address. `role` is the label, such as `Viewer`. `permissionCode` is the rights. `groupId` is the project number, required unless the invite is for an Owner. `createPersonalGroup` true gives Sam a personal folder in that project. A Customer invite must include `startupSceneAssetId`. The invite lasts 7 days. A previous pending invite for the same email in this workspace is revoked. The token in the link is 64 hex characters.

```
POST /workspaces/f0e1d2c3-4a5b-4678-9abc-def012345678/invites
{
  "email": "sam@studio.test",
  "role": "Viewer",
  "permissionCode": "planning.chat|planning.view|scenes.load|sessions.view",
  "groupId": 42,
  "createPersonalGroup": false
}

{
  "id": "d4e5f6a7-1111-4222-8333-444455556666",
  "email": "sam@studio.test",
  "role": "Viewer",
  "rolePreset": "viewer",
  "permissionCode": "planning.chat|planning.view|scenes.load|sessions.view",
  "groupId": 42,
  "createPersonalGroup": false,
  "startupSceneAssetId": null,
  "expiresAt": "2026-11-18T10:40:00.000Z",
  "emailSent": true,
  "inviteUrl": "https://app.skyform.space/invite/a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789"
}
```

Who uses it. The add-person dialog, `space-admin/src/pages/AddUserDialog.tsx`, through `invitesApi.create`. Server: `space-api/src/routes/invites.ts`.

GET `/workspaces/:workspaceId/invites`

The invites Ada has sent that Sam has not accepted yet. Expired ones are marked expired and left out.

```
{
  "invites": [{
    "id": "d4e5f6a7-1111-4222-8333-444455556666",
    "email": "sam@studio.test",
    "role": "Viewer",
    "rolePreset": "viewer",
    "permissionCode": "planning.chat|planning.view|scenes.load|sessions.view",
    "groupId": 42,
    "groupName": "Drawings",
    "createPersonalGroup": false,
    "startupSceneAssetId": null,
    "expiresAt": "2026-11-18T10:40:00.000Z",
    "createdAt": "2026-11-11T10:40:00.000Z",
    "inviteUrl": "https://app.skyform.space/invite/a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789"
  }]
}
```

Who uses it. The people page, `invitesApi.listPending`. Server: `space-api/src/routes/invites.ts`.

POST `/workspaces/:workspaceId/invites/:inviteId/resend`

Sends the same invite again and moves the expiry 7 days forward. No body. The answer is the pending invite plus `emailSent`. If the mail does not go out, the status is 502 and the link is still in the answer so Ada can copy it.

```
POST /workspaces/f0e1d2c3-4a5b-4678-9abc-def012345678/invites/d4e5f6a7-1111-4222-8333-444455556666/resend
{}
```

Who uses it. The people page, `invitesApi.resend`. Server: `space-api/src/routes/invites.ts`.

DELETE `/workspaces/:workspaceId/invites/:inviteId`

Withdraws an invite that is still pending. The link stops working. A personal folder that was prepared for that invite, and never used, is removed.

```
DELETE /workspaces/f0e1d2c3-4a5b-4678-9abc-def012345678/invites/d4e5f6a7-1111-4222-8333-444455556666

{ "id": "d4e5f6a7-1111-4222-8333-444455556666", "removed": true }
```

Who uses it. The people page, `invitesApi.revoke`. Server: `space-api/src/routes/invites.ts`.

GET `/me/invites`

Invites waiting for the signed-in person, matched by their email. No workspace header. The website shows these on the home screen. `acceptPath` is the page Sam opens.

```
{
  "invites": [{
    "id": "d4e5f6a7-1111-4222-8333-444455556666",
    "workspaceId": "f0e1d2c3-4a5b-4678-9abc-def012345678",
    "workspaceName": "Studio",
    "hasLogo": false,
    "role": "Viewer",
    "expiresAt": "2026-11-18T10:40:00.000Z",
    "createdAt": "2026-11-11T10:40:00.000Z",
    "acceptPath": "/invite/a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789"
  }]
}
```

Who uses it. The home screen, `invitesApi.listMine`. Server: `space-api/src/routes/invites.ts`.

GET `/invites/:token`

The public invite page. No sign-in. It says which workspace, who invited Sam, which project, and whether an account with that email already exists. A used, withdrawn, or expired link answers 410.

```
GET /invites/a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789a1b2c3d4e5f64789

{
  "workspace": { "id": "f0e1d2c3-4a5b-4678-9abc-def012345678", "name": "Studio", "slug": "studio", "hasLogo": false },
  "email": "sam@studio.test",
  "role": "Viewer",
  "rolePreset": "viewer",
  "permissionCode": "planning.chat|planning.view|scenes.load|sessions.view",
  "existingUser": false,
  "expiresAt": "2026-11-18T10:40:00.000Z",
  "invitedBy": { "name": "Ada Novak", "email": "ada@studio.test", "hasAvatar": true },
  "group": { "id": 42, "name": "Drawings" }
}
```

Who uses it. `space-admin/src/pages/InviteAcceptPage.tsx`, `invitesApi.preview`. Server: `space-api/src/routes/invites.ts`.

GET `/invites/:token/logo` and `/invites/:token/inviter`

The workspace logo and the inviter's picture, as image bytes, so the invite page can show them before Sam signs in. Missing pictures are 404.

Who uses it. The invite page, `inviteLogoUrl` and `inviteInviterAvatarUrl` in `space-admin/src/lib/invites.ts`. Server: `space-api/src/routes/invites.ts`.

POST `/invites/:token/accept`

Sam joins Studio. If no account exists for that email, the body must include a username and a password of at least 8 characters. If the account already exists, Sam must be signed in as that email, and the body can be empty. The answer includes a fresh sign-in token and the workspace they just joined. `isNewUser` is true when this call created the account.

EXAMPLE, NEW ACCOUNT

```
{ "username": "sam", "password": "correct-horse", "fullName": "Sam Novak" }

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.sam",
  "isNewUser": true,
  "user": { "id": "2b6c8d90-1111-4222-8333-555566667777", "email": "sam@studio.test", "name": "sam", "fullName": "Sam Novak" },
  "workspace": {
    "id": "f0e1d2c3-4a5b-4678-9abc-def012345678",
    "name": "Studio",
    "slug": "studio",
    "role": "Viewer",
    "rolePreset": "viewer",
    "permissionCode": "planning.chat|planning.view|scenes.load|sessions.view",
    "startupSceneAssetId": null
  }
}
```

Signed in as the wrong person, the server answers 403 with `accountMismatch: true` and `invitedEmail: "sam@studio.test"`.

Who uses it. `InviteAcceptPage.tsx`, `invitesApi.accept`. Server: `space-api/src/routes/invites.ts`.

WHICH PROJECTS A PERSON CAN OPEN

POST `/users/:id/add-group`

Gives Sam one more project, and the folders inside it. The body is a list of project numbers, not an object. The server answers 204.

```
POST /users/2b6c8d90-1111-4222-8333-555566667777/add-group
[42]
```

Who uses it. `usersApi.addGroup` in `space-admin/src/lib/users.ts`. Server: `space-api/src/routes/users.ts`.

PUT `/users/:id/group-access`

Replaces the whole list of projects Sam can open. Projects left off the list are taken away, including a personal folder that lived in one of them. The body is the full list of project numbers.

```
PUT /users/2b6c8d90-1111-4222-8333-555566667777/group-access
[42, 57]
```

Who uses it. `usersApi.setGroupAccess`. Server: `space-api/src/routes/users.ts`.

POST `/users/:id/remove-group`

Takes one project away from Sam. The body is the project numbers to remove. The server answers 204.

```
POST /users/2b6c8d90-1111-4222-8333-555566667777/remove-group
[42]
```

Who uses it. `usersApi.removeGroup`. Server: `space-api/src/routes/users.ts`.

GET `/groups/:id/members`

The other direction: who is in project 42, and who in the workspace is not. `members` are in the project. `available` can be added.

```
GET /groups/42/members

{
  "members": [{ "id": "2b6c8d90-1111-4222-8333-555566667777", "name": "sam", "fullName": "Sam Novak", "email": "sam@studio.test" }],
  "available": [{ "id": "7c2a9e14-1b0d-4e6a-9f31-0a8b6c4d2e11", "name": "ada", "fullName": "Ada Novak", "email": "ada@studio.test" }]
}
```

Who uses it. Project member pickers, `groupsApi` in `space-admin/src/lib/groups.ts`. Server: `space-api/src/routes/groups.ts`.

POST `/groups/:id/members`

Adds people to a project. The body is their user ids. They must already belong to the workspace. The server answers 204.

```
POST /groups/42/members
{ "userIds": ["2b6c8d90-1111-4222-8333-555566667777"] }
```

Who uses it. `groupsApi.addMembers`, from `space-admin/src/hooks/useGroups.ts`. Server: `space-api/src/routes/groups.ts`.

DELETE `/groups/:id/members/:userId`

Takes one person out of one project. No body. The server answers 204. Their personal folder under that project goes away.

```
DELETE /groups/42/members/2b6c8d90-1111-4222-8333-555566667777
```

Who uses it. `groupsApi.removeMember`. Server: `space-api/src/routes/groups.ts`.

DELETING AN ACCOUNT

Taking someone out of a workspace is `DELETE /users/:id` above. Deleting the account itself is one of the two calls below. An account that still owns a workspace cannot be deleted until those workspaces are gone. A platform administrator account cannot be deleted this way.

POST `/me/profile/delete-request`

Ada asks to delete her own account. No body. The server emails a 6-digit code and returns a challenge plus a masked address. She must still be signed in for the next call.

```
{}

{ "deleteChallenge": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.delete", "email": "a***@studio.test" }
```

Who uses it. `space-admin/src/components/profile/AccountDeleteDialog.tsx`, `profileApi.requestDeletionCode`. Server: `space-api/src/routes/profile.ts`.

POST `/me/profile/delete-confirm`

Ada sends the challenge and the 6-digit code. The server deletes the account and answers `{ "ok": true }`. The sign-in token stops working.

```
{ "deleteChallenge": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.delete", "code": "482913" }

{ "ok": true }
```

Who uses it. `AccountDeleteDialog.tsx`, `profileApi.confirmDeletion`. Server: `space-api/src/routes/profile.ts`.

GET `/admin/users`

Every account on the platform, for a platform administrator. Each row includes which apps are on, the Drive quota, and how many bytes My Drive uses. `driveQuotaBytes` for a new account is 5368709120, which is 5 GB.

```
{
  "users": [{
    "id": "2b6c8d90-1111-4222-8333-555566667777",
    "email": "sam@studio.test",
    "name": "sam",
    "fullName": "Sam Novak",
    "isPlatformAdmin": false,
    "createdAt": "2026-11-11T10:45:00.000Z",
    "lastLoginAt": "2026-11-11T10:46:00.000Z",
    "lastSeenAt": "2026-11-11T11:02:00.000Z",
    "apps": { "works": true, "connector": true, "plan": true, "drive": true },
    "driveQuotaBytes": 5368709120,
    "driveUsedBytes": 24831
  }]
}
```

Who uses it. The admin accounts screen, `space-admin/src/pages/AdminPage.tsx`, through `adminUsersApi.list`. Server: `space-api/src/routes/adminUsers.ts`.

PATCH `/admin/users/:id`

Turns Works, Connector, and Plan on or off for an account, and sets the Drive quota in bytes. Drive itself stays on. The body must include all three app flags and the quota. A platform administrator's own flags are left as they are.

```
PATCH /admin/users/2b6c8d90-1111-4222-8333-555566667777
{
  "apps": { "works": true, "connector": false, "plan": true },
  "driveQuotaBytes": 5368709120
}

{ "user": { "id": "2b6c8d90-1111-4222-8333-555566667777", "apps": { "works": true, "connector": false, "plan": true, "drive": true }, "driveQuotaBytes":
5368709120 } }
```

Who uses it. The admin accounts screen, `adminUsersApi.update`. Server: `space-api/src/routes/adminUsers.ts`.

DELETE `/admin/users/:id`

Deletes Sam's account. This is the platform administrator doing it, without the email code. Sam's sign-in, memberships, and My Drive go away. If Sam still owns a workspace, the server answers 409 and the account stays. The administrator cannot delete their own account. On success the answer is `{ "ok": true }` and Sam gets an email that the account was removed.

```
DELETE /admin/users/2b6c8d90-1111-4222-8333-555566667777

{ "ok": true }
```

Who uses it. The admin accounts screen, `adminUsersApi.remove`. Server: `space-api/src/routes/adminUsers.ts`, which calls `performAccountDelete`.

ALLOWED VALUES

A file or folder

`kind` is `folder`, `file`, `project`, or `shortcut`. `appKind` is `folder`, `project`, `file`, `scene`, `object`, `material`, `flow`, `configuration`, or `shortcut`. `role` on a file is `viewer`, `editor`, or `owner`. Dates are ISO-8601, such as `2026-11-11T10:04:00.000Z`.

Where it is defined. `StorageNodeDto` in `space-api/src/lib/storage/types.ts`. Drive: `DriveModels.swift`. Website: `space-admin/src/lib/storage.ts`.

A workspace member

`role` on a person is `Owner`, `Manager`, `Creator`, `Viewer`, `Customer`, or `Custom`. `rolePreset` is the same word in lowercase, or null for a custom mix. `permissionCode` is the rights, sorted and joined with `|`. `groupAccess` lists project numbers and names. `startupSceneAssetId` is set for a Customer and otherwise null.

Where it is defined. `serializeMember` in `space-api/src/routes/users.ts`. Website: `space-admin/src/lib/users.ts`.

A sync-log row

`cursor` is the row number as a string, such as `"1843"`. `op` is `create`, `rename`, `move`, `content`, `trash`, `delete`, `restore`, or `share`.

Where it is defined. `StorageChangeDto` in `space-api/src/lib/storage/types.ts`.
